

Algoritmi e Strutture Dati – Parte A – 13/06/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Trovare i limiti asintotici superiore e inferiore più stretti possibili per la seguente famiglia di equazioni di ricorrenza (assumendo che $0 < \delta < 1$):.

$$T(n) = \begin{cases} T(\lfloor \delta n \rfloor) + n^{\delta+1} & n > 1 \\ 1 & n \leq 1 \end{cases}$$

A2 – 1D Board – Punti ≥ 8

Sia data una scacchiera monodimensionale con celle numerate da 1 a n , rappresentata da un vettore booleano A dove le celle con valore **true** sono percorribili e quelle con valore **false** non lo sono. Una pedina parte dalla cella *start* ($1 \leq start \leq n$ e $A[start] = \text{true}$) e può muoversi come segue:

- **forward**: se si trova nella cella x e $x + f \leq n$, può spostarsi nella cella $x + f$;
- **backward**: se si trova nella cella x e $x - b \geq 1$, può spostarsi nella cella $x - b$;
- la mossa è consentita solo se la cella di arrivo è percorribile, cioè $A[\text{cella di arrivo}] = \text{true}$.

Scrivere un algoritmo

int minSteps(**boolean**[] A , **int** n , **int** $start$, **int** f , **int** b)

che restituisca il numero minimo di mosse necessarie per raggiungere la cella 1, o -1 se non è raggiungibile.

Discutere informalmente la correttezza dell'algoritmo proposto e calcolarne la complessità computazionale.

A3 – Stringhe pareggiate – Punti ≥ 8

Una stringa si dice *pareggiata* se per ogni carattere minuscolo che compare in essa è presente anche il corrispondente carattere maiuscolo e viceversa. Per esempio, "aCcA" e "caCA" sono pareggiate, "ABa" non lo è perché contiene il carattere "B" ma non contiene il carattere "b". Per comodità, assumete che esista una funzione `swapcase(item c)` che prende in input un carattere c e

restituisce il carattere corrispondente con l'altro tipo di capitalizzazione (maiuscolo se è minuscolo, e viceversa). Scrivere un algoritmo

int largestPaired(**item**[] S , **int** n)

che restituisce la lunghezza della più lunga sottostringa pareggiata contenuta in S (non sottosequenza: la sottostringa deve essere contigua).

Discutere informalmente la correttezza dell'algoritmo proposto e calcolare la sua complessità computazionale.

Per esempio:

- la più lunga sottostringa pareggiata contenuta in "BaCcAno" è "aCcA" e l'algoritmo deve restituire 4;
- nel caso della stringa "aAa", la più lunga sottostringa pareggiata è la stringa stessa, quindi l'algoritmo deve restituire 3;
- la più lunga sottostringa pareggiata in "torTA" non è "tT" (una sottosequenza), ma la stringa vuota e quindi l'algoritmo deve restituire 0.